

CS598 MP1 Report

Leny Pan, lenypan2

I. Introduction

For this project, we were given data regarding bitcoin for dates between 2010-02-23 and 2018-02-20, ranging from the market prices to the transaction volumes and fees and so forth. Using this data, we were tasked with predicting the market price of bitcoin on a given day using statistical learning.

II. Data Cleaning

The data includes a large amount of features for every day of Bitcoin market transactions. In order, they include:

Date

btc_market_price

btc_total_bitcoins

btc_market_cap

btc_trade_volume

btc_blocks_size

btc_avg_block_size

btc_n_orphaned_blocks

btc_n_transactions_per_block

btc_median_confirmation_time

btc_hash_rate

btc_difficulty

btc_miners_revenue

btc_transaction_fees

btc_cost_per_transaction_percent

btc_cost_per_transaction

btc_n_unique_addresses

btc_n_transactions

btc_n_transactions_total

btc_n_transactions_excluding_popular

btc_n_transactions_excluding_chains_longer_than_100

btc_output_volume

btc_estimated_transaction_volume

btc_estimated_transaction_volume_usd

Notably, many of these parameters are highly correlated – for example, `btc_cost_per_transaction_percent` and `btc_cost_per_transaction`, or `btc_estimated_transaction_volume` and `btc_estimated_transaction_volume_usd`.

I ran an IVF to see which features were co-linear, but that still requires human judgement at the end, anyway. So I'll explain my thought process here:

1. `Btc_estimated_transaction_volume_usd` is
 $\text{btc_estimated_transaction_volume} * \text{market_price}$
2. `Btc_avg_blocks_size` is related to `btc_block_size`
3. `Btc_difficulty` is related to `btc_hash_rate`

As such, I dropped these features, as well as `"btc_orphaned_blocks"` as that seemed to be 0 throughout the entirety of the data.

After this, there were 19 remaining factors.

III. Data formatting for model learning.

The basis for my thinking regarding this process is that the price of bitcoin is based on many factors, but that those factors aren't particularly useful for their day. So I input a "time-lag" feature, where a day would have all the features of yesterday and the day before, totalling 3 days worth of features, including the market prices of those days. Then, I would do an "anti-lag" for the market price of tomorrow's bitcoin – meaning that with 3 days of data, the model would try and aim for the next day's market price, which is what investors generally try and do on a given day. Luckily, pandas includes a "shift" feature which handles edge cases.

IV. Model Learning

For this assignment, I chose to use a LASSO Regression. Ridge Regressions and LASSO Regressions penalize using more predictors, and there are frankly way too many predictors when the time lag is implemented (around 80). LASSO regressions heavily penalize additional features and bring their coefficients down to zero, so it can be a lot simpler instead of having some minor noise from the other features. Additionally, with 80 predictors, cutting down on the number of predictors lets the regression run better with less training data (sub-80 days).

Because the LASSO regression penalizes small coefficients, it ends up overpenalizing features with high numbers. As such, Scikit comes with a normalizing

scalar function (StandardScaler()), which is fitted to the `x_train` and then applied equally to the `x_test`.

Because the purpose of this is to use past data to predict future prices, I did not use a random split or cross validation, to avoid using future data to predict past data. Additionally, I kept the train-test time periods tight and close, so the “culture” surrounding bitcoin trading would be somewhat localized.

V. Results

For example purposes, the training data will be from all of 2015, and the test data will be on all of 2016. Upon running the LASSO regression, here are the features that were removed and the features that were kept:

Removed Features (0 coefficient)	Kept Features
['btc_market_cap', 'btc_blocks_size', 'btc_n_transactions_per_block', 'btc_median_confirmation_time', 'btc_hash_rate', 'btc_cost_per_transaction_percent', 'btc_n_transactions', 'btc_n_transactions_total', 'btc_n_transactions_excluding_popular', 'btc_market_price_lag1', 'btc_total_bitcoins_lag1', 'btc_market_cap_lag1', 'btc_blocks_size_lag1', 'btc_n_transactions_per_block_lag1', 'btc_miners_revenue_lag1', 'btc_cost_per_transaction_percent_lag1', 'btc_n_unique_addresses_lag1', 'btc_n_transactions_total_lag1', 'btc_n_transactions_excluding_popular_lag1', 'btc_n_transactions_excluding_chains_longer_than_100_lag1', 'btc_output_volume_lag1', 'btc_estimated_transaction_volume_lag1', 'btc_market_price_lag2', 'btc_total_bitcoins_lag2', 'btc_market_cap_lag2', 'btc_trade_volume_lag2', 'btc_blocks_size_lag2', 'btc_hash_rate_lag2', 'btc_miners_revenue_lag2', 'btc_cost_per_transaction_lag2', 'btc_n_transactions_lag2', 'btc_n_transactions_total_lag2', 'btc_n_transactions_excluding_popular_lag2', 'btc_n_transactions_excluding_chains_longer_than_100_lag2', 'btc_output_volume_lag2', 'btc_market_price_lag1_lag2', 'btc_total_bitcoins_lag1_lag2', 'btc_market_cap_lag1_lag2', 'btc_blocks_size_lag1_lag2', 'btc_median_confirmation_time_lag1_lag2',	['btc_market_price', 'btc_total_bitcoins', 'btc_trade_volume', 'btc_miners_revenue', 'btc_transaction_fees', 'btc_cost_per_transaction', 'btc_n_unique_addresses', 'btc_n_transactions_excluding_chains_longer_than_100', 'btc_output_volume', 'btc_estimated_transaction_volume', 'btc_trade_volume_lag1', 'btc_median_confirmation_time_lag1', 'btc_hash_rate_lag1', 'btc_transaction_fees_lag1', 'btc_cost_per_transaction_lag1', 'btc_n_transactions_lag1', 'btc_n_transactions_per_block_lag2', 'btc_median_confirmation_time_lag2', 'btc_transaction_fees_lag2', 'btc_cost_per_transaction_percent_lag2', 'btc_n_unique_addresses_lag2', 'btc_estimated_transaction_volume_lag2', 'btc_trade_volume_lag1_lag2', 'btc_n_transactions_per_block_lag1_lag2', 'btc_hash_rate_lag1_lag2', 'btc_miners_revenue_lag1_lag2', 'btc_transaction_fees_lag1_lag2', 'btc_cost_per_transaction_lag1_lag2']

'btc_cost_per_transaction_percent_lag1_lag2', 'btc_n_unique_addresses_lag1_lag2', 'btc_n_transactions_lag1_lag2', 'btc_n_transactions_total_lag1_lag2', 'btc_n_transactions_excluding_popular_lag1_lag2', 'btc_n_transactions_excluding_chains_longer_than_100_lag1_lag2', 'btc_output_volume_lag1_lag2', 'btc_estimated_transaction_volume_lag1_lag2']	
---	--

As you can see, the LASSO model vastly simplifies the model, helping with runtime and clarity. When running with certain years as the training data, generally between 20 and 30 predictors are kept out of the 80. Notably, the “kept” features change with different training sets – with 2014 as the training set, for example, the market price from 2 days ago is kept. But since the market price, arguably a very important feature, is dropped after only 2 days, I think adding additional complexity to the model (4+ days) would only serve to smooth the model.

Here are some examples of how the model performs, showcasing the predicted price versus the actual price:

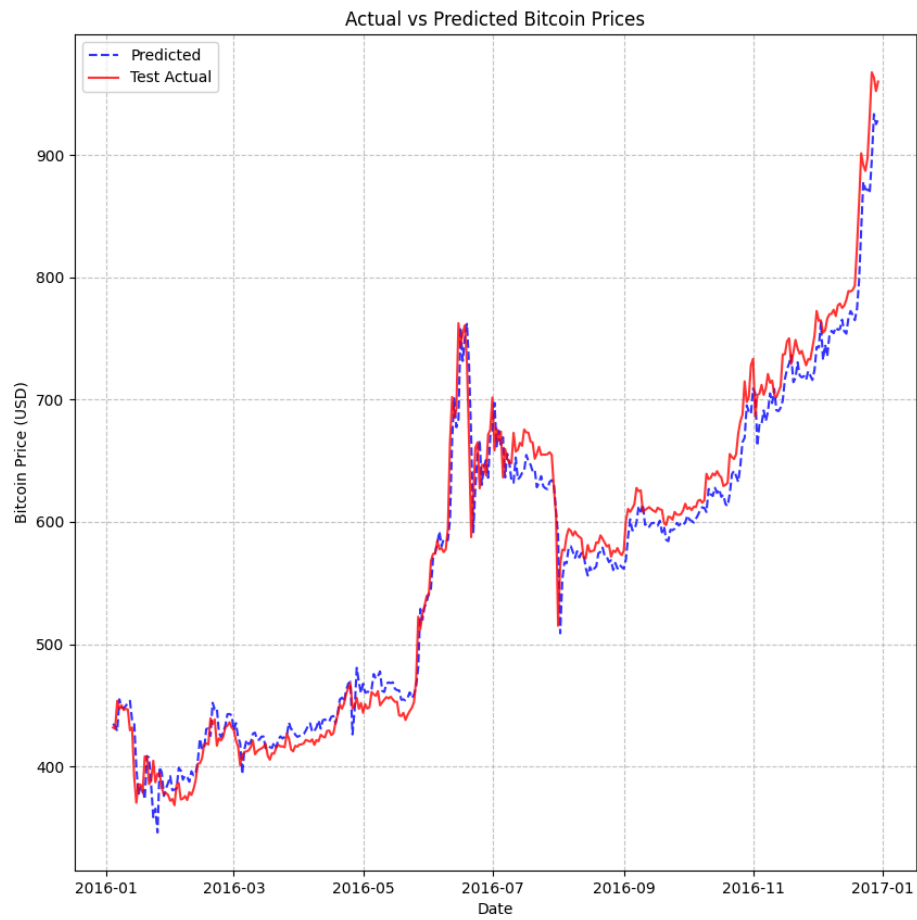


Fig 1: Using 2015 as training data, 2016 as testing data

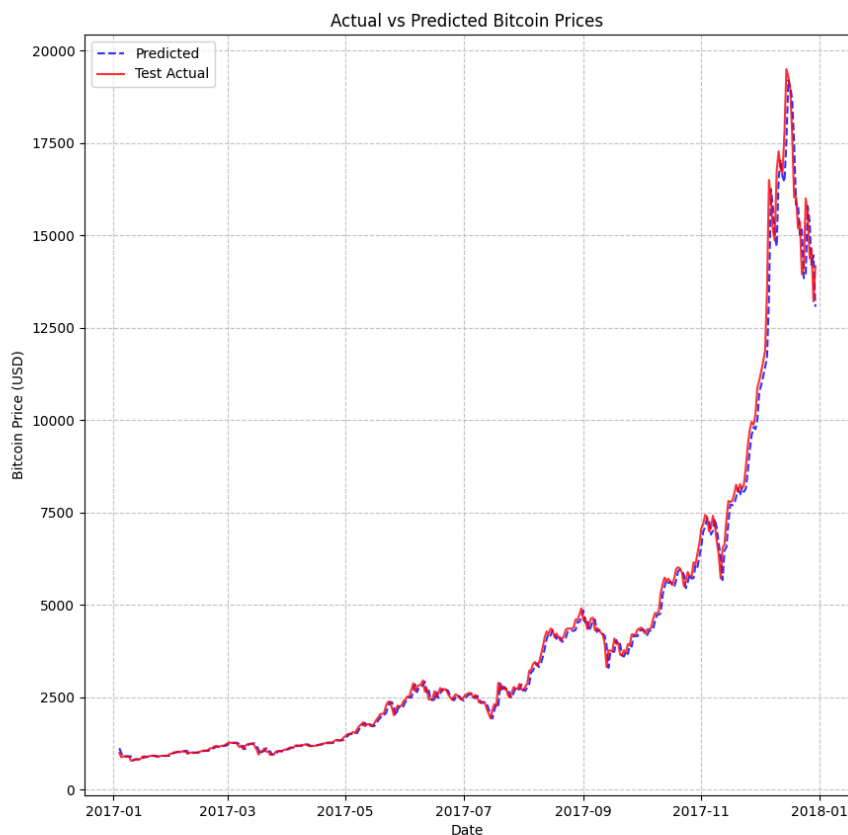


Fig 2: Using 2015-2016 as training data, 2017 as testing data

As you can see, the shape of the regression closely follows that of the actual data, although with a bit of a shift.

Using 2015-training, 2016-test as an example, here are the performance statistics of the model versus a simple linear regression, both including lagged features:

Lasso Model	Simple Linear Regression
Lasso MSE: 433.9996585597917 Model AIC: 2254.441836548485 Model BIC: 2363.4078744796066	OLS MSE: 2053.03802248275 Linear AIC: 2913.0014893014763 Linear BIC: 3208.766449400235

The LASSO model has better AIC and BIC, as well as a significantly lower MSE – around an average of 21 dollars off the price of Bitcoin, which is not too bad considering it's at most 5% of the price – overall, really strong! When done with a 2014-train, 2015-test model, the MSE is only 390, with only 20 predictors – even better. The 2015-2016-train, 2017-test model has an MSE of 136100, around \$369. The first peak in September of that year had a price of 5000, meaning about 7-8% error.